Treap - Group 6

Nguyen Tran Quang Duy Truong Quang Bao Duong
Vo Gia Minh Nguyen Phan Minh Nhat Dang Xuan Phat

1 Introduction and History

1.1 Introduction

A Treap (a portmanteau of Tree and Heap) is a randomized binary search tree data structure. It was designed to maintain a dynamic set of ordered keys while naturally avoiding the unbalanced, worst-case $O(N)$ depth that plagues standard binary search trees when elements are inserted in a sorted or predictable order. Every node within a treap explicitly stores two values: a key (the actual data) and a randomly generated priority.

The structural elegance of a treap comes from satisfying two distinct mathematical properties simultaneously:

- **Binary Search Tree (BST) Property:** With respect to the keys, the tree is a standard BST. For any given node, all nodes in its left subtree have strictly smaller keys, and all nodes in its right subtree have strictly greater keys.
- **Heap Property:** With respect to the priorities, the tree is a max-heap (or min-heap, depending on the implementation). The priority of any parent node must be greater than or equal to the priority of its children.

Because the priorities are assigned randomly upon node creation, the resulting shape of the tree is a random variable. It has the exact same probability distribution as a random binary tree, meaning that with high probability, its height will remain proportional to the logarithm of the number of keys. This guarantees an expected $O(\log_2(N))$ time complexity for search, insertion, and deletion operations, without requiring the complex rotation rules of self-balancing trees like AVL or Red-Black trees.

1.2 History

Original Inception: The treap was first formally proposed and described by computer scientists Raimund Seidel and Cecilia R. Aragon in 1989. They presented it at the 30th Annual Symposium on Foundations of Computer Science (FOCS) and later published a more comprehensive paper in the journal *Algorithmica* in 1996.

The Quicksort Connection: Seidel and Aragon demonstrated that the mathematical shape of a treap mirrors the probabilistic arguments used to prove that the quicksort algorithm runs in expected $O(N \log_2(N))$ time. Essentially, treaps function as "dynamic quicksort," where the randomly generated priorities act as the randomized pivot choices.

The Cartesian Connection: While Seidel and Aragon coined the "Treap," the underlying geometric framework is actually based on the Cartesian Tree, which was introduced a decade earlier by Jean Vuillemin in 1980. A Cartesian tree operates on sequences of points in a 2D plane (x, y). Seidel and Aragon's brilliant historical contribution was realizing that if you take Vuillemin's Cartesian tree and supply randomly generated numbers for the y-coordinates (the priorities), you get a probabilistically balanced search tree.

Later Developments: Following Seidel and Aragon's work, Conrado Martínez and Salvador Roura introduced a related structure called the Randomized Binary Search Tree in 1997, which uses subtree sizes instead of random priority numbers to maintain the same random distribution. Later, in 2000, researchers Naor and Nissim published practical applications of treaps for maintaining authorization certificates in public-key cryptosystems.

1.3 Main Data Structures

- **Standard Treap (Cartesian Tree):** This is the fundamental structure. Every node stores two primary values: a key and a priority. The tree is organized such that the keys satisfy the strict rules of a Binary Search Tree (left descendants are smaller, right descendants are larger), while the priorities satisfy the rules of a Max-Heap (a parent's priority is always greater than or equal to its children's priorities).

- **Implicit Treap:** A structural variation where the explicit search key is removed entirely. Instead, the "key" is the element's implicit index (its relative position in the sequence). This index is dynamically tracked by storing the total size of the subtree at every node.

1.4 Main Algorithms

- **Split:** This foundational algorithm divides a single treap into two separate treaps. For a standard treap, it separates the nodes based on a target key: one resulting treap contains all elements less than or equal to the target, and the other contains all elements greater than the target. For an implicit treap, the split occurs at a specific numerical index.

- **Merge (Join):** The inverse of the split operation. It fuses two distinct treaps into a single valid treap. This requires a strict precondition: every key in the first treap must be strictly smaller than every key in the second treap. The algorithm determines the new root by comparing the random priorities of the two original roots to maintain the heap property, recursively stitching the remaining branches together.

- **Insert:** To add a new element, the algorithm generates a random priority for it. It then splits the existing treap at the exact position where the new element belongs and sequentially merges the resulting left tree, the new node, and the right tree.
- **Erase (Delete):** To remove an element, the algorithm utilizes two consecutive splits to perfectly isolate the target node from the rest of the tree. The isolated node is discarded, and the remaining left and right subtrees are merged to bridge the gap.
- **Bulk Set Operations:** For standard treaps, macroscopic operations like Union, Intersection, and Set Difference are performed recursively by taking the root of the treap with the highest priority, using its key to split the second treap, and merging the corresponding halves.
- **Range Operations (Cut and Paste):** Specific to implicit treaps, this involves performing two splits to slice out a continuous subsegment of the sequence into its own temporary, standalone treap. Once isolated, operations like range queries or full sequence reversals can be applied before merging the segment back into the main structure.

2 Implementation & Complexity

2.1 The Treap Node

A treap node contains several crucial pieces of information: `key`, the value the node is assigned; `prio`, the randomly generated priority of the value; and 2 pointers to `left` and `right` nodes which are its children.

```
struct Node {
    int key;
    int prio;
    Node *left, *right;

    Node(int k) {
        key = k;
        prio = rand();
        left = right = nullptr;
    }
};
```

2.2 Split Function

The split function is a recursive operation that partitions a single Treap into two separate sub-treaps, `l` and `r`, based on a target value `k`. It takes four parameters: the current node pointer `t`, the threshold value `k`, and two pointer references

(`Node*& l` and `Node*& r`) which are updated to point to the roots of the new subtrees containing elements lesser and greater than `k`, respectively.

- **Time complexity:** $O(\log n)$ on average.
- **Space Complexity:** $O(\log n)$ auxiliary space. While no new nodes are created, the recursive calls consume stack space proportional to the height of the tree.

```
void split(Node* t, int k, Node*& l, Node*& r) {
    if (!t) {
        l = r = nullptr;
        return;
    }

    if (t->key <= k) {
        split(t->right, k, t->right, r);
        l = t;
    } else {
        split(t->left, k, l, t->left);
        r = t;
    }
}
```

2.3 Merge Function

The merge function is a recursive operation that joins two separate Treaps, `l` and `r`, into a single unified Treap. It operates under the essential precondition that all keys in the left Treap `l` are smaller than or equal to all keys in the right Treap `r`. The function takes two parameters: the root pointers of the Treaps to be joined.

- **Time Complexity:** $O(\log n)$ on average.
- **Space Complexity:** $O(\log n)$ auxiliary space.

```
Node* merge(Node* l, Node* r) {
    if (!l || !r) return l ? l : r;

    if (l->prio > r->prio) {
        l->right = merge(l->right, r);
        return l;
    } else {
        r->left = merge(l, r->left);
        return r;
    }
}
```

2.4 Insert Function

The insert function is an operation that utilizes the core split and merge functions to add a new key into the Treap while maintaining both BST and Heap properties. It takes two parameters: a pointer reference to the current root `t` (allowing the function to update the tree's entry point) and the new key `k` to be added.

- **Time Complexity:** $O(\log n)$ on average. The function performs one split and two merge operations, each of which traverses a single path from the root to a leaf in a balanced tree.
- **Space Complexity:** $O(\log n)$ auxiliary space.

```
void insert(Node*& t, int k) {
    Node *L, *R;
    split(t, k, L, R);
    Node* newNode = new Node(k);
    t = merge(merge(L, newNode), R);
}
```

2.5 Erase Function

The erase function removes a specific key from the Treap by isolating it through a two-step splitting process. It takes two parameters: a pointer reference to the root `t`, which allows the function to reassign the root after the node is removed, and the key `k` to be deleted. The function first splits the tree into elements less than `k` and elements greater than or equal to `k`. It then performs a second split on the latter group to isolate the exact node `M` containing key `k`. Once isolated, node `M` is deleted from memory, and the remaining left and right Treaps are merged back together.

- **Time Complexity:** $O(\log n)$ on average.
- **Space Complexity:** $O(\log n)$ auxiliary space.

```
void erase(Node*& t, int k) {
    Node *L, *M, *R;
    split(t, k - 1, L, R);
    split(R, k, M, R);

    if (M) {
        delete M;
    }

    t = merge(L, R);
}
```

2.6 Union Function

The Union function is a divide-and-conquer operation used to combine two separate Treaps into a single, balanced Treap. Unlike the standard merge function, `unionTreaps` works even when the keys of the two trees overlap. The function operates by selecting the node with the higher priority to serve as the new root, then recursively splitting the other tree around that root's key to merge their subtrees.

- **Time Complexity:** theoretically $O(m \log(n/m))$, works well in practice with a small hidden constant.
- **Space Complexity:** $O(\log n)$ auxiliary space.

```
Node* unionTreaps(Node* l, Node* r) {
    if (!l || !r) return l ? l : r;

    if (l->prio < r->prio) swap(l, r);

    Node *lt, *rt;
    split(r, l->key, lt, rt);

    l->left = unionTreaps(l->left, lt);
    l->right = unionTreaps(l->right, rt);

    return l;
}
```

3 Applications

3.1 Standard Treap

- **Ease of implementation:** Treap is an easy to implement balanced binary tree.
- **Ordered Set (Order Statistic Tree):** By storing the size of the subtree at each node, you can find the k-th smallest element or the rank of an element in $O(\log n)$.
- **Joining and Splitting Databases:** Unlike Red-Black trees, Treaps can be split into two trees and merged back together in $O(\log n)$ time.

3.2 Implicit Treap

- **Dynamic Array with Fast Insert/Delete:** In a standard array or `std::vector`, inserting an element in the middle takes $O(n)$ time. An implicit treap performs insertions and deletions at any position in $O(\log n)$.

- **Persistent Data Structures:** Because Treaps are based on split and merge operations, they are relatively easy to make persistent. This allows keeping track of all previous versions of the array, which is useful in "Undo" mechanisms in text editors or version control logic.
- **Rope Data Structure:** Implicit treaps are used to manage large blocks of text (often called a Rope data structure).They allow for efficient cutting, copying, and pasting of massive strings without reallocating the entire buffer.

4 Source Codes and Datasets

4.1 Source codes

Access the source code on GitHub:
`https://github.com/MikaelStar639/cs163-treap-seminar/tree/main/programing_assignment`

4.2 Datasets

The datasets have been fully provided in the GitHub repository at the same link above

5 Programming Assignments

5.1 Exercise 1: Game (IOI 2013)

Problem Statement: Bazza and Shazza are playing a game on a massive $R \times C$ grid (up to $10^9 \times 10^9$). Initially, all cells are 0. The system must process two types of operations:

- **Update:** Assign a new non-negative integer K to the cell at row P and column Q.
- **Query:** Calculate the Greatest Common Divisor (GCD) of all integers within a rectangular block from (P, Q) to (U, V).

Detailed Solution:

1. **Data Structure Strategy:** Due to the coordinate range (10^9), a standard 2D array or Segment Tree is impossible. We use a **2D Sparse Structure**: A Sparse Segment Tree for the rows, where each node in the Segment Tree points to a **Treap** (acting as a dynamic 1D structure) for the columns.
2. **GCD Property:** Unlike Sum or Max queries, GCD does not have an inverse operation. Therefore, each Treap node must store the GCD of its subtree.
3. **Operation Execution:**
   - **Update:** Traverse the Segment Tree to find the row range. In each visited node, update the corresponding column in its internal Treap using $O(\log C)$.
   - **Query:** Perform a standard 2D range query. In the Segment Tree, identify nodes covering the row range $[P, U]$, and for each, perform a range GCD query in its Treap for the column range $[Q, V]$.

Complexity Evaluation: The time complexity is $O(Q \cdot \log R \cdot \log C)$ per operation, and the memory complexity is $O(Q \log R)$ as we only create nodes when needed (Sparse/Dynamic allocation).

5.2 Exercise 2: Reversals and Sums (CSES)

Problem Statement: Given an array of n integers, process m operations:

- Reverse a subarray from position a to b.
- Calculate the sum of values in a subarray from a to b.

Detailed Solution:

1. **Implicit Treap Implementation:** Since we need to reverse segments and shift indices, we use an **Implicit Treap** where the "key" is the node's position. Each node stores a `value`, `subtree_sum`, and a `reverse_flag` (Lazy Tag).
2. **Range Reversal (Lazy Propagation):** To reverse $[a, b]$, we split the Treap into three parts: L (positions $1 \ldots a-1$), M ($a \ldots b$), and R ($b+1 \ldots n$). We flip the `reverse_flag` of the root of M. When merging or splitting later, we push this flag down to children, swapping their left and right pointers.
3. **Range Sum:** Similar to reversal, isolate the segment M via two splits and simply return `M->subtree_sum`.

Complexity Evaluation: Each split and merge operation takes $O(\log n)$. Total time complexity is $O(m \log n)$.

5.3 Exercise 3: Substring Reversals (CSES)

Problem Statement: Given a string of length n, process m operations to reverse a substring $[a, b]$. Output the final string.

Detailed Solution:

1. **Implicit Treap for Strings:** This is a variation of the "Rope" data structure. Each node in the Treap stores a single character.
2. **Algorithm:**
   - **Reverse:** Isolate the substring $[a, b]$ using `split`. Toggle the `reverse_flag` on the root of the isolated subtree. Merge everything back.
   - **Output:** After all m operations, perform an in-order traversal of the Treap (remembering to `push` lazy tags) to retrieve the final character sequence.

Complexity Evaluation: The interaction response is highly efficient at $O(m \log n)$, making it feasible for $n, m = 2 \cdot 10^5$, whereas a naive $O(n \cdot m)$ approach would fail.

5.4 Exercise 4: Yaroslav and Points (Codeforces 295E)

Problem Statement: There are n points on the OX axis. Process m queries:

- Move point i to a new coordinate x.
- Calculate the sum of distances between all pairs of points (p_j, p_k) such that $l \leq p_j, p_k \leq r$.

Detailed Solution:

1. **Mathematical Observation:** The sum of distances for a set of points $X = \{x_1, x_2, \ldots, x_k\}$ sorted increasingly is $\sum_{i<j}(x_j - x_i)$. This can be calculated as $\sum(x_i \cdot (2i - 1 - k))$.
2. **Treap Node Information:** Each node in the Treap (sorted by coordinate) stores:
   - `cnt`: Number of points in the subtree.
   - `sum`: Sum of coordinates in the subtree.
   - `ans`: Sum of distances between all pairs within the subtree.
3. **Merging Heuristic:** When merging nodes L and R, the new `ans` is:
$$\text{ans}_{\text{new}} = \text{ans}_L + \text{ans}_R + (\text{cnt}_L \cdot \text{sum}_R - \text{cnt}_R \cdot \text{sum}_L)$$
4. **Update/Query:** Moving a point involves `erase` and `insert` in the Treap ($O(\log n)$). Querying involves `split` to isolate the range $[l, r]$ and returning its `ans`.

Complexity Evaluation: Average time complexity is $O(m \log n)$. This structure avoids expensive $O(N)$ recalibrations for every point movement.

6 Performance Comparison

6.1 Comparison 1: Substring Reversals (CSES)

The Problem: You are given an initial string and a series of range queries $[a, b]$. For each query, the task is to flip the part of the string from position a to position b backwards in place. After all m operations, the final resulting string is printed.

The dataset size n increases from $20,000$ to $200,000$ in increments of $20,000$. Both the brute force and the optimal Treap solution were tested.

Interpretation:

- **Time Efficiency:**
  - **Brute Force ($O(n \cdot m)$):** Execution time grows quadratically. As n and m approach $2 \cdot 10^5$, the time exceeds 7,500 ms, far surpassing the 1.00s time limit.

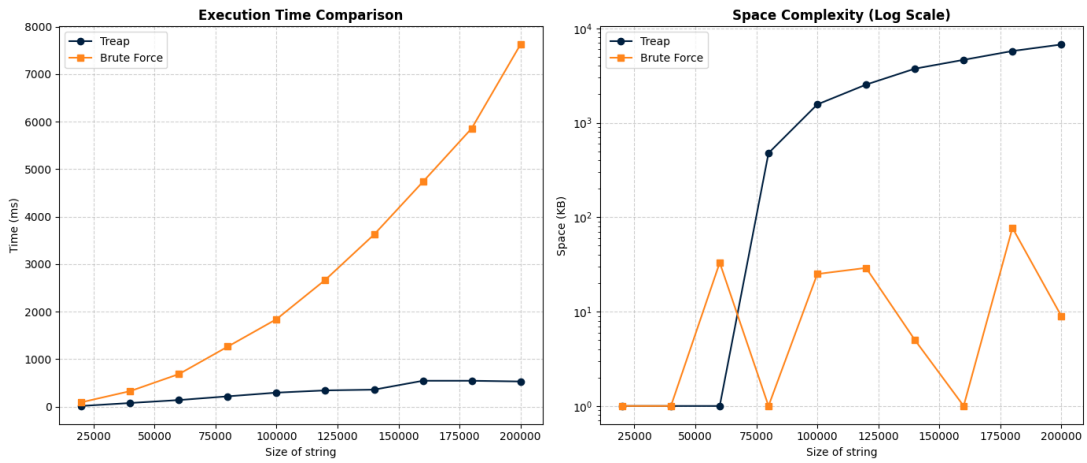

Figure 1: Execution Time Comparison and Space Complexity

  - **Treap ($O(m \log n)$):** Execution time grows logarithmically. Even at maximum input size, it remains under 600 ms, making it the only viable solution.

- **Space Complexity:**
  - **Brute Force:** Maintains a very low memory footprint (under 100 KB) as it modifies the string in place.
  - **Treap:** Shows a significant increase in memory usage, reaching nearly 10 MB, due to the overhead of storing pointers, priorities, and values for every character node.

6.2 Comparison 2: Insertion and Retrieval (BST vs. Treap vs. RBTree)

The Problem: The program takes a permutation of integers from 0 to $n-1$. The process consists of:

1. **Insertion:** Loading all n numbers into the data structure one by one.
2. **Retrieval:** Searching for and retrieving each of those n numbers to confirm existence.

The dataset size n increases from $3,000$ to $30,000$. Each test is run 5 times to obtain an average execution time.

Interpretation:

- **Standard BST:** Performs reasonably well with random data because the tree stays naturally balanced. However, it fails in the worst case

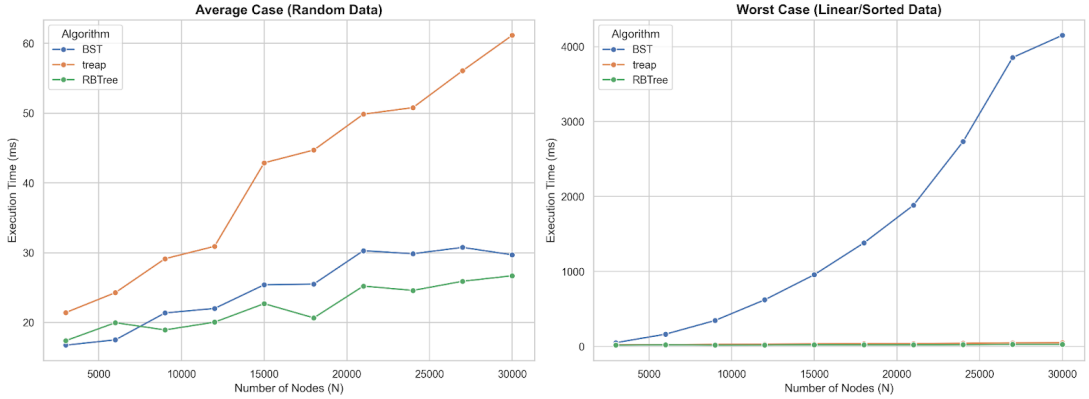

Figure 2: Execution Time Comparison: Average Case vs. Worst Case

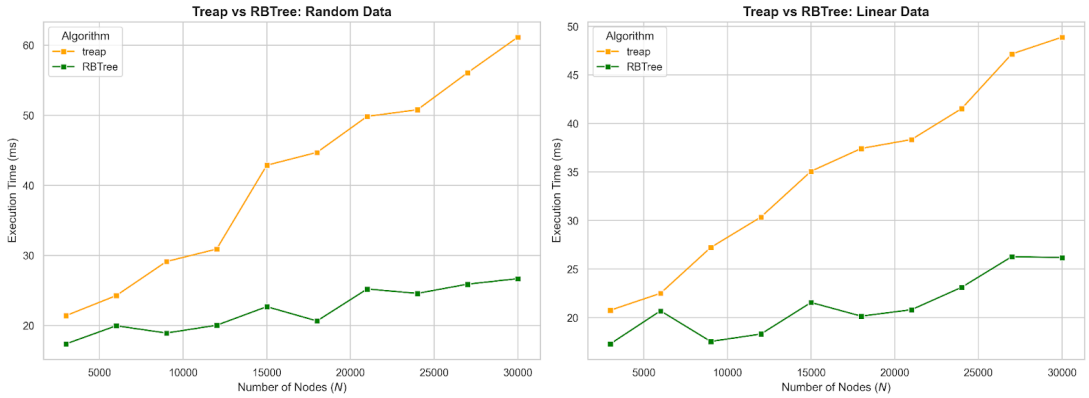

Figure 3: Treap vs RBTree

(sorted data), with execution time spiking to over 4,000 ms at 30,000 nodes, behaving like a linked list.

- **Treap:** Highly consistent regardless of whether data is random or linear, staying under 70 ms. It is slightly slower than others in the average case due to the overhead of generating random priorities and performing rotations.
- **Red-Black Tree:** The most efficient structure in this test, consistently staying around or below 25 ms for both random and linear data.

7 Citations

- Wikipedia Contributors. (2026). Treap. Wikipedia, The Free Encyclopedia.
- Algorithms for Competitive Programming (cp-algorithms). Treap (Cartesian tree).
- USACO Guide. Advanced Data Structures: Treaps.
- TutorialPoints. Treaps in Data Structures.